⚡ DATABRICKS PYSPARK CHEAT SHEET

Production Reference: DataFrame API, Strict Schemas, Joins, Windowing & Performance

PYSPARK | DBR 15.4+ LTS | DATAFRAME API | DELTA LAKE

🚀 SparkSession Initialization & DBR Sizing

Configuration

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
import pyspark.sql.types as T
from pyspark.sql.window import Window as W

spark = (SparkSession.builder
    .appName("Databricks-Production-Pipeline")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.sql.execution.arrow.pyspark.enabled", "true")
    .config("spark.sql.adaptive.enabled", "true")
    .getOrCreate())
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Delta Extensions:** `io.delta.sql.DeltaSparkSessionExtension` registers Delta ACID query parsing and catalog metadata handlers.
- **Catalog Delegation:** `DeltaCatalog` integrates local and Unity Catalog 3-level namespaces for ACID transactions.
- **PyArrow Optimization:** `arrow.pyspark.enabled` enables zero-copy vectorized serialization between JVM and Python worker processes.
- **AQE Enabled:** Activates Adaptive Query Execution for dynamic runtime join conversions and automatic partition coalescing.

⚡ **Serverless Note:** On Databricks Serverless compute, the SparkSession is pre-initialized and instantly auto-tuned for sub-second scaling.

📥 DataFrame Projections, Transforms & Filters

DataFrame API

```
df_curated = (df_raw
    # Filter with pushdown predicate
    .filter((F.col("status") == "ACTIVE") & (F.col("amount") > 0))
    # Multi-column projection and derivation
    .withColumn("tax_amt", F.round(F.col("amount") * 0.18, 2))
    .withColumn("customer_tier",
        F.when(F.col("amount") >= 10000, "PLATINUM")
         .when(F.col("amount") >= 2500, "GOLD")
         .otherwise("STANDARD"))
    # High-performance projection replacing select() + withColumn()
    .selectExpr("customer_id", "amount", "tax_amt", "customer_tier",
                "current_timestamp() as ingested_at")
)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Filter Pushdown:** `.filter(...)` generates a Catalyst Filter operator pushed down into Parquet footer metadata to skip non-matching row groups.
- **F.when().otherwise():** Compiles directly into native JVM conditional bytecode; avoids python UDF context switching.
- **selectExpr():** Evaluates mixed column projections and SQL expressions in a single projection pass without intermediate plans.
- **withColumns() in Spark 3.3+:** Pass a dictionary to add multiple derived columns simultaneously without rebuilding schema trees.

💡 **Optimization:** Chain transformations lazily; Catalyst combines consecutive filter and project operators into a single pipelined stage.

📐 Strict Schema Definition (StructType & StructField)

Data Quality

```
# Explicit schema avoids expensive inferSchema full-scans
txn_schema = T.StructType([
    T.StructField("txn_id", T.LongType(), nullable=False),
    T.StructField("cust_id", T.StringType(), nullable=False),
    T.StructField("amount", T.DecimalType(18, 2), nullable=False),
    T.StructField("attributes", T.MapType(T.StringType(), T.StringType())),
    T.StructField("items", T.ArrayType(T.StringType())),
    T.StructField("event_time", T.TimestampType())
])

# Read with enforced schema and corrupt record safety
df = (spark.read
      .schema(txn_schema)
      .option("mode", "PERMISSIVE")
      .option("columnNameOfCorruptRecord", "_corrupt")
      .json("abfss://landing@adls.dfs.core.windows.net/txns/"))
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Explicit Schema:** Eliminates multi-pass storage scans required by `inferSchema=True`, reducing multi-terabyte ingestion time by 80%.
- **Nullable Guarantees:** Setting `nullable=False` enables Catalyst to optimize join plans and bypass redundant null-check branches.
- **PERMISSIVE Mode:** Captures malformed input JSON rows safely into a quarantine column without crashing distributed streaming tasks.
- **Nested Types:** `MapType` and `ArrayType` preserve schema structure while enabling downstream element-level indexing.

⚡ **FinOps Sizing:** Enforcing schemas on read eliminates full file body scans across cloud storage landing zones.

🧰 pyspark.sql.functions (F) Master Reference

Functions

```
df_transformed = (df
    # String transforms
    .withColumn("full_key", F.concat_ws("_", F.col("region"), F.col("cust_id")))
    # Date formatting & truncation
    .withColumn("month_start", F.date_trunc("MM", F.col("txn_date")))
    # Null coalescing
    .withColumn("clean_phone", F.coalesce(F.col("phone"), F.lit("UNKNOWN")))
    # In-memory array deduplication
    .withColumn("unique_tags", F.array_distinct(F.col("tags")))
    # JSON string extraction
    .withColumn("device_id", F.get_json_object(F.col("raw_meta"), "$.device.id"))
)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **JVM Native Execution:** Built-in `pyspark.sql.functions` map directly to Catalyst expressions executed in JVM/C++ Photon without Py4J bridge overhead.
- **concat_ws():** Joins strings using a delimiter and skips null arguments automatically, preventing unwanted null outputs.
- **date_trunc():** Truncates timestamps to the requested calendar boundary, accelerating time-series aggregations.
- **get_json_object():** Extracts scalar JSON path attributes using high-speed JSON path scanning without parsing entire payload schemas.

⚡ **Vectorization Rule:** Never implement custom Python UDFs when equivalent native `F.` functions exist.

🔀 Joins, Broadcast Hints & Skew Salting

Optimization

```
# Broadcast Join: Eliminates shuffle for lookup tables (<100MB)
df_joined = df_fact.join(F.broadcast(df_dim), on="cat_id", how="left")

# Semi & Anti Joins (Replaces expensive IN / NOT IN subqueries)
df_active = df_users.join(df_purchases, on="user_id", how="left_semi")
df_churned = df_users.join(df_purchases, on="user_id", how="left_anti")

# Manual 2-Stage Skew Salting (When AQE cannot balance 100M+ key)
df_skewed = df_fact.withColumn("salt", (F.rand() * 10).cast("int"))
df_dim_exploded = (df_dim
    .withColumn("salt_arr", F.array([F.lit(i) for i in range(10)]))
    .withColumn("salt", F.explode("salt_arr")))
df_salted_join = df_skewed.join(df_dim_exploded, on=["key", "salt"])
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Broadcast Hash Join (BHJ):** Replicates the small dimension to all executors, eliminating the expensive SortMergeJoin shuffle exchange.
- **Left Semi Join:** Evaluates existence without duplicating parent rows when child records match multiple times.
- **Left Anti Join:** Returns records from the left dataset having zero matches on the right; highly optimized alternative to `NOT IN`.
- **Synthetic Salting:** Distributes skewed hot keys across 10 virtual partition buckets, eliminating straggler tasks holding up the stage.

⚡ **Threshold Knob:** Configure `spark.sql.autoBroadcastJoinThreshold = 104857600` (100MB) to tune auto-broadcast eligibility.

🪟 Analytical Window Specifications (pyspark.sql.Window)

Analytics

```
# Define Window Specifications
w_dept = W.partitionBy("dept_id").orderBy(F.col("salary").desc())
w_roll_7d = (W.partitionBy("account_id")
              .orderBy(F.col("txn_date").cast("timestamp").cast("long"))
              .rangeBetween(-7 * 86400, 0)) # 7 days in seconds

df_analyzed = (df
    .withColumn("dense_rank", F.dense_rank().over(w_dept))
    .withColumn("prev_salary", F.lag("salary", 1, 0).over(w_dept))
    .withColumn("salary_diff", F.col("salary") - F.col("prev_salary"))
    .withColumn("rolling_7d_total", F.sum("amount").over(w_roll_7d))
)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **W.partitionBy().orderBy():** Directs Spark to sort rows within partitioned subsets without sorting the entire distributed dataset.
- **rangeBetween(offset, 0):** Computes rolling aggregate windows across epoch timestamps, properly accounting for days with missing records.
- **F.dense_rank():** Computes ranking without rank number gaps for tied values; ideal for identifying top-earners per department.
- **F.lag(col, offset, default):** Inspects prior record values in a single streaming pass across the partition window.

💡 **Range vs Rows:** `rangeBetween` handles continuous value ranges; `rowsBetween` operates strictly on physical row indices.

⚡ DATABRICKS PYSPARK CHEAT SHEET

Delta API, Auto Loader Streaming, Arrow UDFs, Partitioning, Storage & Microbatching

DELTA LAKE API | AUTO LOADER | VECTORIZED UDFS

DeltaTable Python API & Upsert

Delta API

```python
from delta.tables import DeltaTable

delta_tbl = DeltaTable.forName(spark, "silver.customers")

(delta_tbl.alias("t")
 .merge(df_updates.alias("s"), "t.cust_id = s.cust_id")
 .whenMatchedUpdate(condition="s.hash != t.hash", set={
     "email": "s.email",
     "status": "s.status",
     "updated_at": "current_timestamp()"
 })
 .whenNotMatchedInsert(values={
     "cust_id": "s.cust_id", "email": "s.email",
     "status": "s.status", "created_at": "current_timestamp()"
 })
 .execute())
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **DeltaTable.forName():** Binds programmatic Python handles directly to Unity Catalog registered tables without raw path references.
- **Declarative Merge:** Supports conditional updates and inserts; compiles into an atomic ACID commit updating the Delta transaction log.
- **Hash Comparison:** Skips rewriting unchanged rows, minimizing write amplification on Delta Deletion Vectors and Parquet storage.
- **Schema Evolution:** Pass `.withOption("mergeSchema", "true")` to automatically append new columns arriving from source DataFrames.

Deletion Vectors: In DBR 14.1+, enable Deletion Vectors (`delta.enableDeletionVectors = true`) to execute row updates without file rewrites!

Structured Streaming with Auto Loader (cloudFiles)

Streaming Ingestion

```python
df_stream = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "abfss://chk@adls.dfs.core.windows.net/schem
a")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .option("cloudFiles.inferColumnTypes", "true")
    .load("abfss://raw@adls.dfs.core.windows.net/landing/"))

query = (df_stream.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "abfss://chk@adls.dfs.core.windows.net/chk_stream")
    .trigger(availableNow=True)
    .toTable("bronze.events_stream"))
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Auto Loader (cloudFiles):** Incrementally discovers and processes new cloud files without expensive file listing calls using cloud notifications.
- **Schema Evolution:** `addNewColumns` safely adapts to evolving JSON payloads without throwing runtime streaming validation errors.
- **Checkpoint Location:** Guarantees exactly-once processing state by persisting commit offsets and metadata in durable cloud storage.
- **trigger(availableNow=True):** Processes all pending backlogged data across microbatches, then automatically terminates compute for FinOps savings.

FinOps Note: `availableNow=True` achieves streaming exactly-once semantics with scheduled batch infrastructure costs.

Vectorized Apache Arrow UDFs (@pandas_udf)

Vectorization

```python
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf(T.DoubleType())
def calculate_vat(amount: pd.Series, rate: pd.Series) -> pd.Series:
    # SIMD Vectorized NumPy/Pandas calculation in Python worker
    return (amount * (rate / 100.0)).round(2)

df_with_tax = df.withColumn("vat", calculate_vat(F.col("amount"), F.col("tax_rate")))
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Apache Arrow Zero-Copy:** Transmits column chunks between JVM and Python workers in Apache Arrow columnar format without row serialization.
- **Vectorized Execution:** Python calculations execute on whole Pandas Series using vectorized C/SIMD loops instead of slow row-by-row iteration.
- **Throughput Speed:** Executes up to **100x faster** than standard Python `@udf`, eliminating Py4J bottleneck latency.
- **Memory Batching:** Tune batch sizes via `spark.sql.execution.arrow.maxRecordsPerBatch` (e.g. 10,000) to balance RAM usage.

Type Contract: Python type hints `(pd.Series) -> pd.Series` determine the vectorization execution model in PySpark 3.0+.

Partitioning Strategy (repartition vs coalesce)

Partitioning

```python
# repartition(N): Full shuffle, balances partitions uniformly
df_shuffled = df.repartition(200, "region")

# repartitionByRange: Sorts data across partitions for range queries
df_ranged = df.repartitionByRange(50, F.col("txn_date").desc())

# coalesce(N): Merges partitions locally WITHOUT full network shuffle!
df_collapsed = df_shuffled.coalesce(10)

# Inspect partition distribution
print("Partition Count:", df_collapsed.rdd.getNumPartitions())
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **repartition():** Executes a full cluster network shuffle; essential when increasing partitions or redistributing skewed datasets.
- **repartitionByRange():** Uses sampling to determine partition split points, producing globally sorted outputs across partition boundaries.
- **coalesce():** Merges adjacent physical partitions within executors without network transfer; only use when decreasing partition count.
- **File Target Sizing:** Adjust partition counts prior to writing Delta tables to maintain 100MB-1GB target Parquet files.

Anti-Pattern: Calling `coalesce(1)` on large datasets collapses upstream execution to a single executor, triggering Driver OOM!

Storage Levels & Memory Persistence

Memory Management

```python
from pyspark import StorageLevel

# Deserialized in RAM (Fastest CPU, Highest RAM)
df.cache() # Shorthand for persist(StorageLevel.MEMORY_AND_DISK_DESER)

# Serialized in RAM with Disk Spill (Production Recommended for Large DFs)
df.persist(StorageLevel.MEMORY_AND_DISK_SER)

# Trigger materialization with an action
record_count = df.count()

# Crucial: Unpersist when downstream processing completes!
df.unpersist(blocking=True)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **cache() vs persist():** `cache()` defaults to deserialized objects in memory; `persist()` allows customizing serialization and disk spill.
- **MEMORY_AND_DISK_SER:** Stores serialized byte arrays; reduces RAM consumption by ~3x and prevents out-of-memory errors via local disk spill.
- **Lazy Evaluation:** Persisting is registered lazily; data is only populated into cache blocks when the first action (e.g. `.count()`) is invoked.
- **unpersist(blocking=True):** Explicitly frees cluster memory immediately, preventing memory leaks across long-running pipelines.

Off-Heap Option: `StorageLevel.OFF_HEAP` stores data in Tungsten managed memory outside the JVM garbage collector, eliminating GC pauses!

Streaming Microbatch Merges (foreachBatch)

Streaming Sink

```python
def upsert_microbatch(microbatch_df, batch_id):
    # Idempotent write per microbatch
    (DeltaTable.forName(spark, "silver.realtime_orders").alias("t")
     .merge(microbatch_df.alias("s"), "t.order_id = s.order_id")
     .whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute())

query = (df_stream.writeStream
    .format("delta")
    .foreachBatch(upsert_microbatch)
    .option("checkpointLocation", "abfss://chk@adls.dfs.core.windows.net/orders_chk")
    .start())
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **foreachBatch:** Bridges Structured Streaming and standard batch APIs, allowing arbitrary Delta MERGE logic per microbatch partition.
- **Idempotent Commits:** Using `batch_id` guarantees exactly-once processing even if a microbatch fails and replays from the checkpoint.
- **Multi-Table Fan-Out:** A single stream can write to multiple Delta tables or external databases simultaneously within the microbatch function.
- **whenMatchedUpdateAll():** Dynamically syncs all columns between source and target without requiring explicit column-by-column mapping.

Streaming Rule: Cache `microbatch_df` inside `foreachBatch` if reading it multiple times to prevent re-evaluating the streaming plan.